

Software Requirements Specification

Multi-tenant University ERP

Academic • Marketing & Finance
Administration & Human Resources

Document identifier	ERP-SRS-001
Version / status	1.1 / Group review draft
Baseline	Backend 6e10c54; frontend 6cc6859; reviewed 13 September 2026
Course	SEN4121 – Large System Environment, Summer 2026
Instructor	Engr. Tanwi Nkiamboh
Scope	React frontend and four-service backend, with deployment and integration boundaries
Prepared by	Four-member Software Engineering project group

Group members

No.	Full name	Student / registration ID
1		
2		
3		
4		

Reading guide

Six-page main narrative followed by detailed acceptance criteria, a requirement catalogue, use cases, verification traceability, diagrams and submission checklists. Read the implementation-status legend before treating a requirement as delivered.

This document describes observed code, specified requirements and unverified targets separately. It is not a certification of production readiness, statutory payroll compliance or successful execution of application tests.

Document control

Version	Date	Change
1.0	13 September 2026	Initial code-grounded specification with editable PlantUML sources and local PDF build.
1.1	13 September 2026	Assignment alignment, four-member submission forms and clarified UML execution/relationship notation.

Review and approval

Responsibility	Name / signature	Date
Product / academic representative	_____	_____
Finance and payroll representative	_____	_____
Technical lead / document reviewer	_____	_____
Course submission approval	_____	_____

Authority and interpretation

The supplied four-page SEN4121 project brief is the source of stakeholder obligations. Source code, migrations and tests determine what is implemented. Existing operational reports are historical evidence, not rerun results. The ISO/IEC/IEEE 29148 reference is used as an organizational alignment for requirements; no conformance assessment or reproduction of that standard is claimed.

Group submission responsibilities

All four members must review and understand both documents. Complete the individual contribution statements, obtain all four signatures on each, and sign the submission declaration supplied in the technical-report appendices. The report's source acknowledgments form part of the submission and defense. Blank names, signatures and evidence fields must be completed by the group; this review draft records no approval or individual contribution.

Pagination policy

The SRS main narrative is six pages; the companion SDD main narrative is eight pages. Main narrative pages are deliberately bounded; extensive requirements, interface inventories and diagrams are appendices. The brief limits the combined SRS and technical report to 20 pages excluding appendices and diagrams. The package budgets six SRS narrative pages, eight SDD narrative pages and three front-matter pages per document: 20 combined non-appendix pages including covers, control pages and contents. Appendices are explicitly labeled and excluded from that total; this is not two separate 20-page allowances.

Contents

Document control	i
1 Purpose, scope and stakeholder goals	1
2 Product context, actors and constraints	2
3 Functional scope and current delivery boundary	3
4 Roles, tenancy and information rules	4
5 Interfaces and business workflow contracts	5
6 Quality requirements and quantitative acceptance	6
A Verification strategy and acceptance gates	7
B Assumptions, risks and unresolved decisions	8
C Detailed requirements catalogue	9
D Detailed use-case specifications	22
E Requirement-to-verification traceability	25
F Use-case and context diagrams	27
G Glossary and submission checklist	30

1 Purpose, scope and stakeholder goals

1.1 Purpose

This Software Requirements Specification (SRS) establishes the review baseline for the ICT University multi-tenant Enterprise Resource Planning system. It states what the system must do, its boundaries, observable acceptance conditions and unresolved obligations. The companion Software Design Document (SDD) explains how the current implementation realizes those requirements.

1.2 Product scope

The product integrates three business domains through four FastAPI services: Identity; Academic; Finance & Marketing; and Administration & HR. A shared Nginx gateway exposes versioned HTTP APIs. A PostgreSQL cluster hosts four separately owned databases. RabbitMQ carries the enrollment-to-invoice workflow. The React frontend is maintained in the sibling `ERP_System` repository. Both repositories were inspected; the baseline distinguishes functional self-service screens from administrative placeholders.

Stakeholder	Need	Observable benefit
Student	Enrollment, course registration, results, tuition payment	Own records visible; published results and receipts downloadable.
Instructor / staff	Teaching operations, attendance and grades	Assigned-offering work with prerequisite and scheduling checks.
Institution administrator	Domain administration and oversight	Institution-scoped configuration, approvals and reports.
Finance / marketing staff	Fees, expenses, campaigns	Traceable payment settlement and FCFA reporting.
HR administrator / employee	Recruitment, leave, payroll, assets	Auditable decisions and employee self-service.
Platform operator	Deployability and operational control	Independent service images, health checks and recovery procedures.
Assessment panel	Defensible engineering evidence	Requirements linked to source, test assets and UML views.

1.3 Success criteria

Demonstrate one complete asynchronous business transaction across modules; tenant and role isolation; reliable financial recording; the required domain workflows; test evidence; and reproducible deployment/documentation. Production readiness additionally requires the unresolved release gates in Section A, not merely a successful local demonstration.

Requirement authority

The supplied SEN4121 brief defines the mandatory scope. Where the code implements only part of that scope, both the obligation and the gap are retained. An implemented handler is not evidence that every concurrency, security, UI or operational acceptance condition has passed.

2 Product context, actors and constraints

2.1 Boundary and external systems

Browsers call one gateway origin. The frontend supplies role-gated self-service screens and 401-driven session restoration; the backend remains authoritative for authorization. The mobile-money integration is an external provider adapter, not a fifth ERP service. Jenkins, a container registry, host TLS termination and an off-host backup repository belong to the operating environment rather than to business modules. Only the registry and TLS configuration intentions are represented in deployment assets; a functioning production environment is not established by these documents.

2.2 Mandatory constraints

1. At least three independently deployable business services, each owning its database/schema. The baseline uses four databases, with Identity as the fourth service.
2. An API gateway performs routing, authentication delegation and rate limiting.
3. At least one cross-module workflow uses asynchronous broker messaging.
4. Dockerfiles and Compose orchestration support the full system; Kubernetes is optional.
5. A CI pipeline runs tests and builds images on main-branch changes; its repository trigger and controller configuration require deployment evidence.
6. Relational persistence is designed toward 3NF; intentional snapshots and cross-service references must be explained. Money is whole XAF (FCFA), not binary floating point.
7. Requirements, UML, test plan, coverage evidence and a third-party setup guide accompany the source. The combined main report limit is addressed by the pagination policy.

2.3 Actor semantics

Roles are `super_admin`, `admin`, `staff` and `student`. “Instructor”, “finance officer” and “HR manager” are business responsibilities, not additional implemented role enum values. Staff privileges are broad in some modules; fine-grained departmental permissions are not implied. An HR employee is a separate domain record whose optional identity link is necessary for self-service. An institution is the tenant boundary; campus is descriptive context, not a proven separate campus-level authorization boundary.

2.4 Operating assumptions

The reviewed backend targets Python 3.13 container images, PostgreSQL 17 and RabbitMQ 3.13. It relies on a working sibling frontend build, secure key provisioning, database migrations, a fee schedule for each relevant program/term and linked employee accounts. Initial institution/program/term/user provisioning is primarily through seed or administrative operations, not complete CRUD APIs. Internet access is necessary only for external provider and deployment dependencies, not for communication among local ERP containers.

3 Functional scope and current delivery boundary

Area	Present in source	Outstanding / qualified
Identity	Argon2 login, signed access tokens, refresh rotation, lockout and logout	No MFA, password-reset or full user-management API; access JWTs are not immediately revoked on logout.
Academic	Enrollment, course create/list, prerequisite graph, registrations, attendance, grades, appeals, exams, PDF reports	Full course/program CRUD absent; capacity is stored but not enforced in registration; instructor UI is a placeholder.
Finance	Fee schedules, event-driven invoices, payment intents, reconciliation, receipts, balanced ledger, expenses	Provider adapter not live-sandbox validated; callback is blocked for unauthenticated external callers by current gateway rules.
Marketing	Campaign and lead records, conversion, ROI	Simple paid-invoice attribution; repeated lead associations may count the same revenue more than once.
HR	Recruitment/hiring, shifts and QR tokens, leave decisions, in-app notifications, performance, payroll, stock movements	General employee CRUD/account linking absent; rate attestation is manual; email/SMS and biometric devices absent.
Operations	Compose, CI and promotion definitions, monitoring script, local PITR drill	Live CI/CD, encrypted off-host scheduled backup, centralized log aggregation and target load validation incomplete.

3.1 Requirement status and priority

Implemented (I) means a directly corresponding implementation exists; it does not mean a test was executed during document generation. **Partial (P)** means part of the required behavior exists but a stated acceptance condition, integration boundary or evidence remains missing. **Target (T)** means a requirement is not established by the reviewed baseline. **Must** is necessary for the brief or for safe operation; **Should** is a proposed improvement; **Optional** is outside mandatory baseline scope.

3.2 Numbering and change control

Requirement identifiers in Appendix C are stable review handles. Changing an identifier's meaning requires a revision note, impacted design views, API/event compatibility assessment and updated acceptance evidence. A proposed feature must not be reclassified as implemented merely because it appears in a diagram. Stakeholders must approve new quantitative targets and retention periods.

4 Roles, tenancy and information rules

4.1 Effective authorization baseline

Capability	Implemented restriction / qualification
Enrollment creation	Student: self; admin/staff: explicit student; super admin is not in this creation allow-list.
Course, prerequisite and offering creation	Admin, staff, super admin, with an institution context.
Course registration	Student only, using own enrollment and compatible program/term.
Teaching mutations and risk roster	Admin/super admin or the offering's assigned staff instructor.
Published student results	Student sees only own published grades; appeals require own published grade.
Fees, expenses, campaigns	Admin/staff/super admin for creation and staff functions; fee schedules are not admin-only.
Invoice/payment/receipt self-service	Students restricted to own invoices; other authenticated roles operate within their tenant scope.
Recruitment, assets, payroll, leave decisions	Admin/super admin; payroll schedule verification is super-admin-only.
HR self-service	Staff/admin/super admin with a matching employee/user link; no student access.
Institution-wide access	Tenant claim is verified; a tenantless super-admin context can enable platform RLS access but does not make every tenant-required mutation usable.

4.2 Data rules

Tenant-owned rows carry `institution_id`. Each service validates JWT claims itself, and sets transaction-local database context. Identity credential resolution and finance background tasks deliberately use platform context. Outbox/inbox tables are outside ordinary tenant RLS. Identity enables RLS on users and refresh sessions, not on institution/campus metadata. The bootstrap makes service roles database owners; migrations use forced RLS. This baseline must not be described as separate non-owner runtime and migration credentials.

User, campus, student, program and enrollment identifiers crossing service boundaries are opaque UUID references, not distributed foreign keys. Existence and lifecycle consistency are therefore not guaranteed by the receiving database. Institution scope is mandatory for ordinary mutations. Time instants should be timezone-aware; calendar dates and monthly periods must not be confused with instants. Current monthly summary cutoff follows the worker host's date.

4.3 Sensitive information

Credentials, refresh tokens, personal identifiers, salaries, grades and mobile numbers require least-privilege access. Log redaction, retention, consent, data export and erasure policies require explicit institutional approval; no jurisdictional compliance certification is asserted.

5 Interfaces and business workflow contracts

5.1 HTTP contract

External prefixes are `/api/v1/auth`, `/api/v1/academic`, `/api/v1/finance` and `/api/v1/hr`. JSON request/response schemas are defined with Pydantic. Binary report endpoints return `application/pdf`. UUID identifiers are strings on the wire. Common responses include 200/201, 204 logout, 400 business-input rejection, 401 unauthenticated, 403 forbidden, 404 absent or deliberately concealed object, 409 business conflict, 422 schema validation and 429 gateway budget exhaustion. Detail payloads are not uniformly structured; clients must not assume every error is a single string.

Non-production FastAPI instances expose local `/docs`, `/redoc` and `/openapi.json`. These are not automatically published through the gateway. Production disables them when the effective environment is actually production. The SDD contains a generated inventory of all service-local routes, not a fabricated OpenAPI export.

5.2 Session contract

Login returns a ten-minute access JWT and a rotating, `HttpOnly` refresh cookie. The refresh session defaults to fourteen days and `SameSite=Strict`. Secure-cookie behavior is configuration-dependent. Base Compose explicitly sets `REFRESH_COOKIE_SECURE=false`; the production override does not reset that inherited setting. Correct production configuration and TLS are release gates. Logout revokes the presented refresh session, while already issued access tokens remain valid until expiry. The frontend keeps access tokens in memory, deduplicates refresh calls within one JavaScript runtime and redirects after an unrecoverable 401. It has no proactive expiry timer or cross-tab refresh coordination.

5.3 Asynchronous contract

An accepted enrollment and its outbox row commit atomically. The academic worker publishes `academic.enrollment_accepted.v1` to `erp.events`. The finance consumer uses the envelope tenant and fee schedule to create an invoice. Delivery is at least once; inbox uniqueness and enrollment-level invoice uniqueness limit duplicate effects. A missing fee schedule leads to a dead-letter queue, not a zero-value guessed invoice. HTTP enrollment success therefore does not promise immediate invoice availability.

5.4 Payment and notification contract

The backend persists a payment reference before contacting the provider. Provider status, not callback claims, authorizes settlement. Polling is the viable baseline when provider callbacks cannot cross the current gateway policy. HR notifications are persisted records retrieved by polling; email, SMS and push delivery are not implemented.

6 Quality requirements and quantitative acceptance

Quality attribute	Acceptance target	Available evidence / limit
Performance	25 concurrent sessions, 5 minutes, read p95 below 500 ms, errors below 1%; exclude provider latency.	Historical local 90-second smoke: p95 882.9 ms, 0.48% errors (3/621). Latency target not met; full duration not run.
Reliability	Enrollment is not lost if broker is unavailable; duplicate delivery has one invoice effect.	Outbox/inbox and transaction design; restart and outage evidence must accompany acceptance.
Recoverability	Proposed operational objectives: RPO at most 5 min, RTO at most 60 min.	Tiny isolated PITR drill reports 21.8 s RTO; not a production estimate or complete ERP recovery.
Quality gate	At least 70% core business-logic coverage, unit/integration/contract checks.	CI config sets coverage gate at 70%; actual current reports are not generated by this documentation task.
Security	Tenant/role/ownership checks; short-lived credentials; mitigation evidence for at least five OWASP categories.	Code and security report exist; production configuration and end-to-end validation remain necessary.
Observability	Liveness/readiness per service; useful logs, broker backlog and failure alerts.	Basic logging and scheduled checks; centralized aggregation and backup-age alerting not established.
Scalability	Document replica strategy without losing session/transaction correctness.	Stateless APIs and worker locking are design enablers; no HA or measured scaling claim.

6.1 Test conditions

Performance acceptance must record host specification, dataset size, duration, traffic mix, authentication model, pacing and source-IP distribution. The existing k6 script shares one login token and source IP and measures four read routes. Its pacing is necessary to avoid measuring only the gateway rate limiter; 25 VUs are not automatically evidence for 25 independently authenticated user sessions.

Availability percentages, maximum database size, notification latency, accessibility level and legal retention periods have not been approved. They are intentionally not invented here. Correctness, isolation and explicit error behavior take precedence over optimistic uptime or throughput claims.

A Verification strategy and acceptance gates

A.1 Verification methods

Inspection verifies source, migrations and contracts. **Analysis** checks formulas, normalization and dependency boundaries. **Test** exercises valid, invalid, cross-tenant, duplicate and concurrent paths. **Demonstration** executes the full browser/gateway/broker/database workflow. Appendix E maps requirements to existing test files and additional acceptance scenarios. Those scenarios are specifications, not newly executed tests.

A.2 Entry conditions

Use synthetic tenants and users with all four roles; provision migrated service databases and signing keys; create matching program/term/fee schedules; register students in course offerings; link employees to identity users; select a deterministic payment test double or an explicitly authorized sandbox. Never execute destructive recovery drills against a shared production database.

A.3 Exit conditions

1. All Must requirements are implemented or have an explicit, instructor-approved exception. Partial and target items remain visible in the submission.
2. Required tests pass against the identified commit with stored reports and coverage evidence. Critical cross-module flows are demonstrated through the gateway.
3. Load results meet approved conditions or are honestly reported as a failed gate.
4. Payroll rates have authoritative, period-specific review; example data is not used for real payments. Provider integration is accurately labeled mock or sandbox.
5. A representative recovery test verifies all databases, events, sessions and financial invariants. Scheduled off-host backup is established before real records.
6. Effective production Compose settings, TLS, secrets, image publication, CI triggers and promotion workflow are verified in the actual target environment.

A.4 Defect and evidence management

Record requirement ID, expected/observed behavior, reproduction, severity, owner, commit and retest evidence for each issue. Existing documentation names unresolved work but is not a replacement for the required issue tracker board. The release authority must approve documented exceptions; this draft does not grant them.

Evidence integrity

Only document and diagram builds are executed as part of this deliverable. Historical performance and recovery values are quoted with their limitations. No application test pass count, coverage percentage, production deployment or sandbox transaction is claimed without a corresponding run artifact.

B Assumptions, risks and unresolved decisions

ID	Risk / open decision	Resolution required
R01	Full program/course CRUD and general employee management exceed exposed APIs.	Agree missing API scope and lifecycle rules; retain as mandatory gaps.
R02	Production overlay inherits development environment/cookie settings and may merge port bindings.	Inspect effective Compose configuration; verify loopback exposure, production mode and Secure cookies.
R03	Callback handler is public locally but gateway requires bearer authentication.	Explicitly design provider callback ingress or approve polling-only integration.
R04	Rate schedules are not legal proof; sample values are unverified.	Attach current CNPS/DGI sources, period, calculation convention and signed review.
R05	Single host, shared database cluster and broker are common failure points.	Capacity plan; off-host backups; recovery rehearsal; defer HA claims.
R06	Source and legacy operational prose disagree on runtime database ownership.	Use migrations/bootstrap as baseline; separately approve any credential separation.
R07	CI builds but does not push images; promotion assumes images already exist.	Implement/verify artifact publication and immutable release pairing before promotion.
R08	Recent-two-grade mean is not the same as two consecutive individual failures.	Approve the implemented at-risk rule or revise it through change control.
R09	Centralized logging, EXPLAIN comparisons and full-duration load evidence missing.	Collect representative artifacts, with timestamps and environment details.
R10	Frontend is self-service focused; administrative screens and analytics are placeholders.	Complete required instructor/admin dashboards; verify cache isolation on account changes and missing UI error states.

B.1 Excluded unless separately approved

MFA is optional under the brief. Kubernetes, multi-region availability, real biometric hardware, automated bank disbursement, tax filing, refunds/installments, email/SMS gateways, predictive machine learning and general-ledger completeness are not delivered features. Some may be future enhancements, but they must not appear as implemented actors or flows in the baseline UML.

B.2 Scope closure

The following appendices provide the detailed contractual catalogue, scenarios, verification matrix and supporting diagrams. They expand the specification without changing the six-page main narrative. Review status is deliberately unsigned.

C Detailed requirements catalogue

All acceptance conditions below are to be verified at the stated scope. Evidence paths are repository-relative. A path identifies implementation or test material, not a claim that it passed. Source origins: AUTH/DATA/INT/OPS derive from brief section A; ACAD/FIN/HR derive from section B; DOC from sections C/D and the checklist.

C.1 Identity, access and tenant context

C.1.1 AUTH-01 – Secure login

Status: **Implemented** **Priority:** Must

Requirement. The identity service shall authenticate active accounts using Argon2 password verification and return an RS256 access token only after successful verification.

Acceptance. Correct credentials return 200 and token metadata; incorrect or inactive-account credentials return 401. Password plaintext shall not be persisted. Email lookup uses the normalized lowercase value.

Evidence: `services/identity/app/routers/auth.py`

C.1.2 AUTH-02 – Token lifetime and claims

Status: **Implemented** **Priority:** Must

Requirement. Access tokens shall have a default lifetime of ten minutes and carry identity, role and tenant/-campus context; resource services shall independently verify signatures, issuer, audience and expiry.

Acceptance. Expired, wrong-key, wrong-issuer and wrong-audience tokens return 401 at protected routes. A spoofed tenant header shall not override a valid token's tenant.

Evidence: `services/identity/app/core/security.py`

C.1.3 AUTH-03 – Refresh rotation

Status: **Implemented** **Priority:** Must

Requirement. A valid refresh request shall revoke the used refresh row, create a replacement in the same family and return a new access token; only refresh-token hashes shall be stored.

Acceptance. Sequential reuse of an already revoked token returns 401 and revokes the remaining active family. A replacement token works before expiry. Concurrent-refresh behavior requires a separate race test; it is not inferred from sequential coverage.

Evidence: `services/identity/tests/test_auth.py`

C.1.4 AUTH-04 – Lockout

Status: **Implemented** **Priority:** Must

Requirement. Five failed password checks shall trigger a default fifteen-minute account lockout; successful login after the permitted time shall reset the failure state.

Acceptance. Attempts during the lockout return 401 even with correct credentials. Verify exact threshold and recovery using controlled time; the gateway has an independent login budget.

Evidence: `services/identity/app/core/config.py`

C.1.5 AUTH-05 – Logout and client expiry

Status: **Partial** **Priority:** Must

Requirement. Logout shall revoke the presented refresh session and clear its cookie. The UI shall end an unauthenticated session when tokens can no longer be refreshed.

Acceptance. Logout returns 204. Subsequent refresh is rejected. Existing access tokens remain valid until their own expiry; immediate access-token revocation is not promised. The frontend implements 401-driven refresh/redirect; test network-error logout and multiple tabs separately before declaring complete session-lifecycle acceptance.

Evidence: `services/identity/app/routers/auth.py`

C.1.6 AUTH-06 – Role and ownership enforcement

Status: **Partial** Priority: Must

Requirement. At least four roles shall be enforced at API and UI layers, with object ownership checks for student and employee self-service.

Acceptance. Run the Section 4 role matrix across two tenants. Forged object identifiers must not reveal another student's invoices or employee's private records. API checks and page-level role checks exist. Navigation remains visible to all authenticated roles; unsupported role/module combinations display placeholders rather than full administrative screens.

Evidence: `services/hr/app/deps.py`

C.1.7 AUTH-07 – Tenant-scoped persistence

Status: **Implemented** Priority: Must

Requirement. Ordinary tenant requests shall use verified institution context and forced RLS on scoped business and Identity user/session tables; service database roles shall not connect to other service databases.

Acceptance. A tenant-A request cannot read/write tenant-B rows in protected tables; each runtime role fails a connection attempt to another service database. Outbox/inbox tables, Identity institution/campus metadata and platform-context flows are explicit exceptions.

Evidence: `tests/integration/test_db_isolation.py`

C.1.8 AUTH-08 – Production session configuration

Status: **Partial** Priority: Must

Requirement. Production browser sessions shall use HTTPS and Secure, HttpOnly, SameSite refresh cookies; the deployment configuration shall not inherit development overrides.

Acceptance. Inspect the effective merged deployment, perform a browser login over HTTPS and inspect cookie attributes. This is not currently established by the production override, which leaves base environment and cookie settings in place.

Evidence: `docker-compose.prod.yml`

C.1.9 AUTH-09 – Administrative MFA

Status: **Target** Priority: Optional

Requirement. If adopted, administrators shall perform a second authentication factor before privileged access, with recovery and audit controls.

Acceptance. An approved factor, enrollment/recovery policy and positive/negative tests are required before activation. No OTP or MFA endpoint exists in the baseline.

Evidence: `services/identity/app/routers/auth.py`

C.2 Academic management

C.2.1 ACAD-01 – Program and course lifecycle

Status: **Partial** Priority: Must

Requirement. Authorized staff shall manage programs and courses through create, read, update and delete/archive operations that preserve referenced academic records.

Acceptance. Program list and course create/list exist. Complete program creation/update/removal and course update/removal require implementation and lifecycle acceptance; seed scripts shall not be presented as complete CRUD APIs.

Evidence: `services/academic/app/routers/courses.py`

C.2.2 ACAD-02 – Enrollment acceptance

Status: **Implemented** **Priority:** Must

Requirement. A student shall enroll only themselves; admin/staff may enroll a specified student. Acceptance shall atomically persist the enrollment and a billing event outbox row.

Acceptance. Valid same-tenant program/term returns 201; missing references return 404; repeat student/program/term returns 409 in the normal duplicate path. Rolled-back acceptance shall leave neither record. Campus/student external existence is not cross-database verified.

Evidence: `services/academic/app/routers/academic.py`

C.2.3 ACAD-03 – Prerequisite graph

Status: **Implemented** **Priority:** Must

Requirement. Course prerequisites shall reject self-reference and cycles before insertion.

Acceptance. Add A requires B, B requires C; C requires A must return 409. Existing edges must not duplicate. Rejected edges shall leave the graph unchanged. Concurrent cycle creation remains a separate verification scenario.

Evidence: `services/academic/app/routers/courses.py`

C.2.4 ACAD-04 – Course registration

Status: **Implemented** **Priority:** Must

Requirement. Student registration shall require their own enrollment, matching program and term, and a published average of at least 50 percent in an offering of every prerequisite.

Acceptance. A score average of 50 passes; an average below 50, unpublished-only results or no passing offering fails. Duplicate registration returns 409 in the ordinary path. Capacity is stored but is not an enforced acceptance rule in the baseline.

Evidence: `services/academic/tests/test_courses.py`

C.2.5 ACAD-05 – Teaching assignment scope

Status: **Implemented** **Priority:** Must

Requirement. Teaching mutations shall be limited to admin/super-admin or the assigned staff instructor of the target offering.

Acceptance. Assigned instructor can manage assessments/attendance/exams; unrelated staff and students receive 403. All checks use the offering from the current tenant.

Evidence: `services/academic/app/core/authz.py`

C.2.6 ACAD-06 – Academic attendance

Status: **Implemented** **Priority:** Must

Requirement. Authorized teaching staff shall create attendance sessions and mark student presence.

Acceptance. Create session, record present/absent marks and retrieve results. Current mark entry does not validate course-registration membership. Summary calculations count all created sessions, with missing presence marks contributing no attendance credit.

Evidence: `services/academic/tests/test_attendance.py`

C.2.7 ACAD-07 – Grades and publication

Status: **Implemented** **Priority:** Must

Requirement. Teaching staff shall enter assessment scores between zero and the assessment maximum, and explicitly publish results before student visibility.

Acceptance. Out-of-range score returns 400. Student sees only own published scores. Published scores are still editable through the ordinary grade-entry handler; immutable grade history is not a delivered general guarantee.

Evidence: `services/academic/app/routers/grades.py`

C.2.8 ACAD-08 – Grade appeals

Status: **Implemented** **Priority:** Must

Requirement. A student shall appeal an owned published grade. Reviews shall transition from submitted to under-review, accepted or rejected, and from under-review to a final decision.

Acceptance. Reject a second open appeal, invalid transition or final-state reopening. Acceptance requires a bounded corrected score; a changed score writes an audit row in the same transaction. An unchanged accepted score need not add a score-change row.

Evidence: `services/academic/tests/test_grades.py`

C.2.9 ACAD-09 – Exam scheduling

Status: **Implemented** **Priority:** Must

Requirement. Scheduling and rescheduling shall reject overlapping room, instructor or registered student usage within an institution/term and serialize conflict checks for that scope.

Acceptance. Use overlapping intervals to obtain 409 for each resource conflict. Adjacent intervals are permitted. Start must precede end. A term advisory lock prevents two concurrent same-term schedules from both passing conflict checks. Cross-term conflicts are not checked.

Evidence: `services/academic/app/routers/exams.py`

C.2.10 ACAD-10 – Reports and at-risk indicator

Status: **Implemented** **Priority:** Must

Requirement. The system shall export transcript and attendance-summary PDFs and compute a documented at-risk flag for registered learners.

Acceptance. Flag when attendance is below 0.75 OR the mean of the latest two published assessment percentages is below 50. Fewer than two grades yields no grade-average signal; no sessions yields no attendance signal. Boundaries 0.75 and 50 are not flagged by that signal. Students may export only their own reports.

Evidence: `services/academic/app/routers/reports.py`

C.2.11 ACAD-11 – Academic dashboards

Status: **Partial** **Priority:** Must

Requirement. Students and instructors shall receive current, role-appropriate dashboards using real enrollment, grade, attendance and at-risk information.

Acceptance. Student course cards show live registrations, grades and at-risk values and provide report downloads. Instructor management and dynamic instructor dashboards remain placeholders; the overview page only shows a welcome message.

Evidence: `services/academic/app/routers/reports.py`

C.3 Finance and marketing

C.3.1 FIN-01 – Fee schedules

Status: **Implemented** **Priority:** Must

Requirement. Finance shall configure a nonnegative whole-XAF fee for an institution/program/term tuple.

Acceptance. Authorized admin/staff/super-admin creates one schedule; duplicate tuple returns 409. Academic references remain opaque IDs. The consumer must use the matched schedule, never a caller-provided or default price.

Evidence: `services/finance/app/routers/finance.py`

C.3.2 FIN-02 – Asynchronous invoice creation

Status: **Implemented** **Priority:** Must

Requirement. Enrollment events shall produce one invoice effect per institution/enrollment, with replay handling by event and business identity.

Acceptance. Deliver the same event twice, then a distinct event ID for the same enrollment; verify one invoice. Missing fees must dead-letter without persisting the inbox success marker. A transient failure must requeue rather than acknowledge a lost invoice.

Evidence: `services/finance/tests/test_consumer.py`

C.3.3 FIN-03 – Invoice visibility

Status: **Implemented** **Priority:** Must

Requirement. Students shall list and retrieve only their own tenant-scoped invoices, with pending or paid status and amount in XAF.

Acceptance. A student requesting another student's invoice receives 404; list filters exclude it. Enrollment filtering narrows a list without overriding ownership.

Evidence: `services/finance/app/routers/finance.py`

C.3.4 FIN-04 – Payment initiation

Status: **Implemented** **Priority:** Must

Requirement. Payment initiation shall persist a unique provider reference before requesting payment, and allow at most one pending/succeeded attempt per invoice.

Acceptance. Non-pending invoices and a second in-progress intent return 409. A provider-call failure leaves a pending intent with a logged error rather than falsely marking paid. Polling checks status; it does not guarantee resubmission of a request never accepted by MTN.

Evidence: `services/finance/app/routers/payments.py`

C.3.5 FIN-05 – Atomic settlement

Status: **Implemented** **Priority:** Must

Requirement. Confirmed provider success shall atomically mark the intent succeeded and invoice paid, post balanced cash-debit/tuition-revenue-credit entries and create a receipt.

Acceptance. Repeat and concurrently trigger reconciliation; observe one settlement effect and one receipt. Inject a transaction failure to verify no partially paid ledger state. Failed provider status shall not create a receipt or revenue entry.

Evidence: `services/finance/app/reconciliation.py`

C.3.6 FIN-06 – Mobile-money integration

Status: **Partial** **Priority:** Must

Requirement. The payment portal shall integrate with an approved mobile-money sandbox or a clearly labeled simulated gateway, with status verification independent of callback content.

Acceptance. The deterministic test double is acceptable under the brief. The MTN adapter requires real credentialed sandbox evidence before being called validated. External callback delivery requires a gateway policy decision; a service-local unauthenticated handler is insufficient.

Evidence: `services/finance/app/payments/mtn_momo.py`

C.3.7 FIN-07 – Digital receipts

Status: **Implemented** **Priority:** Must

Requirement. A settled invoice shall have a downloadable PDF receipt scoped to permitted viewers.

Acceptance. A paid invoice returns a nonempty PDF with the recorded amount and reference; an unpaid invoice shall not yield a success-shaped receipt. Student ownership checks shall also apply to the report endpoint.

Evidence: `services/finance/tests/test_receipt.py`

C.3.8 FIN-08 – Expenses and financial ledger

Status: **Implemented** **Priority:** Must

Requirement. Authorized staff shall record expenses and a balanced expense-debit/cash-credit ledger posting in one transaction.

Acceptance. Nonnegative whole-XAF expense produces a matching balanced transaction; unbalanced ledger input is rejected by the posting helper. Students cannot create/list staff expenses. This is a limited ledger, not a complete statutory accounting suite.

Evidence: `services/finance/app/routers/expenses.py`

C.3.9 FIN-09 – Campaign and lead tracking

Status: **Implemented** **Priority:** Must

Requirement. Authorized staff shall create/list campaigns, create/list their leads and associate converted leads with student UUIDs.

Acceptance. Reject a body/path campaign mismatch; missing campaign returns 404. Conversion updates the lead to converted. A complete CRM contact profile and irreversible conversion workflow are not part of the current lead model.

Evidence: `services/finance/app/routers/campaigns.py`

C.3.10 FIN-10 – Campaign ROI

Status: **Partial** **Priority:** Must

Requirement. Campaign analytics shall expose recorded cost, attributed paid tuition and ROI, explicitly representing the undefined zero-cost case.

Acceptance. For cost C greater than zero and attributed revenue R , ROI is $(R-C)/C$. For $C=0$, ROI is null with a reason. Current join attribution may duplicate revenue for repeated lead/student links; deduplicated attribution needs a business-rule decision.

Evidence: `services/finance/tests/test_campaigns.py`

C.3.11 FIN-11 – Monthly summaries

Status: **Implemented** **Priority:** Must

Requirement. The system shall generate monthly XAF summaries from posted tuition revenue and expenses, retain new versions on regeneration and catch up missing elapsed months.

Acceptance. Revenue/expense totals use a half-open monthly interval and net equals their difference. Re-generation increments version, without replacing history. Catch-up creates missing version-1 periods since first ledger activity, not the current incomplete month.

Evidence: `services/finance/app/summaries.py`

C.3.12 FIN-12 – Finance and marketing dashboards

Status: **Partial** **Priority:** Must

Requirement. Role-appropriate UI dashboards shall show actual payment, expense, monthly-summary and campaign metrics, including pending and unavailable data.

Acceptance. The student invoice/payment/receipt screen exists. Non-student Finance renders a placeholder: expense, campaign and analytics screens are absent. Do not count pending invoices as paid revenue or coerce null ROI to zero when these dashboards are added.

Evidence: `services/finance/app/routers/summaries.py`

C.4 Administration and human resources

C.4.1 HR-01 – Recruitment and hiring

Status: **Implemented** **Priority:** Must

Requirement. Authorized HR administrators shall create positions/candidates and atomically hire a candidate by creating an employee record and setting the candidate to hired.

Acceptance. Hiring returns 201 and a new employee; a candidate already marked hired returns 409. Identity provisioning is separate and the new employee's user link is null. The generic stage-update endpoint does not enforce a recruitment transition graph.

Evidence: `services/hr/app/routers/recruitment.py`

C.4.2 HR-02 – Employee lifecycle

Status: **Partial** **Priority:** Must

Requirement. The system shall manage employee profiles, employment state and identity linkage needed for attendance, leave, performance and payroll.

Acceptance. Persistence and hire creation exist. General employee list/edit, termination workflows and identity-link administration require additional implementation or an approved scoped exception; they shall not be inferred from recruitment routes.

Evidence: `services/hr/app/models/employees.py`

C.4.3 HR-03 – QR attendance

Status: **Implemented** **Priority:** Must

Requirement. HR administrators shall create shifts and issue ten-minute signed QR tokens; employees shall check in only to their linked shift using the current token.

Acceptance. Expired, replaced or forged tokens return 401; another employee's shift returns 403; a repeated shift/employee check-in returns 409. The QR token uses a separate secret/audience. Physical presence, device attestation and shift-time-window enforcement are not guaranteed.

Evidence: `services/hr/tests/test_attendance.py`

C.4.4 HR-04 – Leave requests and decisions

Status: **Implemented** **Priority:** Must

Requirement. Employees shall submit their own dated leave requests; authorized HR administrators shall approve/reject only pending requests and shall not decide their own leave.

Acceptance. End before start returns 422; self-decision returns 403; a second decision returns 409. Successful decision records reviewer/time and persists a notification for the linked user.

Evidence: `services/hr/app/routers/leave.py`

C.4.5 HR-05 – Task notifications

Status: **Implemented** **Priority:** Must

Requirement. Leave decisions shall produce in-application notifications for linked employee accounts and allow each user to list and mark only their notifications read.

Acceptance. Decision and notification commit together. Another user's notification ID returns 404. Mark-read records a timestamp once. Email, SMS, push and streaming delivery are excluded.

Evidence: `services/hr/app/routers/leave.py`

C.4.6 HR-06 – Performance reviews

Status: **Implemented** **Priority:** Must

Requirement. HR administrators shall record performance reviews with employee, reviewer, period, rating and comments; employees shall retrieve their own reviews.

Acceptance. Missing employee returns 404; student access is rejected; own-review list is based on the authenticated user's linked employee, not a caller-supplied employee ID.

Evidence: `services/hr/tests/test_performance.py`

C.4.7 HR-07 – Versioned payroll rates

Status: **Partial** **Priority:** Must

Requirement. Payroll shall use versioned CNPS, standard-deduction and marginal IRPP schedules, with authoritative published rates verified for the effective period before approval.

Acceptance. New schedules start unverified; only a super-admin can attest verification. Capture official source, date, units and boundary examples before using real payroll. The boolean verification flag alone is not statutory validation or immutable provenance.

Evidence: `services/hr/app/routers/payroll.py`

C.4.8 HR-08 – Payroll calculation

Status: **Implemented** **Priority:** Must

Requirement. Payroll shall compute capped employee/employer CNPS, taxable base, marginal tax and net pay with Decimal arithmetic and whole-XAF half-up rounding.

Acceptance. Test gross zero, negative rejection, the CNPS ceiling, every tax boundary and half-franc rounding. Employer contribution shall not reduce employee net. The SDD gives the exact implemented calculation order; legal parameter suitability is separate.

Evidence: `services/hr/tests/test_payroll_calc.py`

C.4.9 HR-09 – Payroll runs and approval

Status: **Implemented** **Priority:** Must

Requirement. One draft payroll run per institution/period shall snapshot payslips for active employees and shall be approved only when its schedule is verified.

Acceptance. A duplicate period returns 409. Unverified schedule prevents approval with 403. Approval stores actor/time; repeated approval returns 409. No salary disbursement or automatic Finance ledger integration is performed.

Evidence: `services/hr/tests/test_payroll_run.py`

C.4.10 HR-10 – Assets and inventory

Status: **Implemented** **Priority:** Must

Requirement. Asset movements shall distinguish employee assignments/returns from general-stock adjustments and serialize updates to protect aggregate available stock.

Acceptance. Under the asset row lock, reject aggregate assignment above stock, total negative assignment, or adjustment below assigned stock. Assignment delta positive means units issued; negative means return. Per-employee return balances are not independently enforced.

Evidence: `services/hr/app/routers/assets.py`

C.4.11 HR-11 – HR dashboards

Status: **Partial** **Priority:** Must

Requirement. An HR dashboard shall summarize leave state and performance analytics, and support employee visibility of their own attendance, reviews and payslips.

Acceptance. The staff workspace includes leave, manual QR-code entry, approved payslips and notifications. Admin/super-admin receive a placeholder, and performance analytics, recruitment, asset and payroll-approval screens are absent.

Evidence: `services/hr/app/routers/performance.py`

C.5 Data, integration and operational quality

C.5.1 DATA-01 – Service-owned relational data

Status: **Implemented** **Priority:** Must

Requirement. Each service shall own its database and migrations, with local referential constraints, tenant indexes and a documented normalized design.

Acceptance. Inspect the four databases and least-privilege connection tests. Explain intentional snapshot/JSON structures and opaque external IDs; do not claim distributed foreign keys or a formal complete 3NF proof from an ERD alone.

Evidence: `ops/postgres/init/01-init-databases.sh`

C.5.2 DATA-02 – Query optimization evidence

Status: **Partial** **Priority:** Must

Requirement. High-read queries shall have justified indexes and before/after query-plan evidence on a representative dataset.

Acceptance. Capture EXPLAIN ANALYZE with buffer statistics for tenant invoice lookup, outbox polling and report aggregation, preserving dataset/query/result comparability. Existing indexes are not by themselves evidence that the load target is met.

Evidence: `services/finance/app/models/finance.py`

C.5.3 DATA-03 – Backup and recovery

Status: **Partial** **Priority:** Must

Requirement. Scheduled encrypted off-host backups and continuous WAL archival shall support a documented, tested point-in-time recovery procedure.

Acceptance. Restore all four service databases from a representative backup, validate consistency and reconcile broker/session state. The isolated plain-Postgres drill is partial evidence; pgBackRest scheduling, retention, remote storage and complete ERP rehearsal remain outstanding.

Evidence: `docs/backup-recovery.md`

C.5.4 INT-01 – Versioned REST API and static registry

Status: **Implemented** **Priority:** Must

Requirement. Services shall expose versioned JSON APIs, document schemas/errors using OpenAPI and be discoverable through a documented service registry.

Acceptance. Check each route prefix, schema and error behavior. Compose DNS names plus explicit Nginx upstreams from the static registry. Export service OpenAPI from a non-production instance; do not assume gateway forwarding of local documentation routes.

Evidence: gateway/nginx.conf

C.5.5 INT-02 – Gateway controls

Status: **Implemented** **Priority:** Must

Requirement. The gateway shall delegate protected-request authentication and apply per-client-IP budgets of 20 requests/second generally and 5 login requests/minute.

Acceptance. Verify 429 responses beyond configured bursts (40 general, 5 login). Client identity headers shall be replaced by delegated identity, and direct service JWT validation shall remain active. Public exceptions must match the gateway configuration.

Evidence: gateway/auth_request.conf

C.5.6 INT-03 – Event contract evolution

Status: **Partial** **Priority:** Must

Requirement. Cross-service events shall have stable IDs, tenant/correlation metadata and explicit schema versions, with compatibility validation before promotion.

Acceptance. Validate checked-in examples against JSON Schemas. Test rejection of unsupported event type/version at runtime before claiming strict version enforcement: the present consumer parses required fields but does not fully apply the JSON Schema or version dispatch.

Evidence: contracts/test_examples_match_schema.py

C.5.7 OPS-01 – Containerized clean setup

Status: **Partial** **Priority:** Must

Requirement. A third party shall be able to configure, build, migrate, seed and start the complete system using documented prerequisites and Compose.

Acceptance. Use a clean machine and sibling frontend checkout; verify all service readiness and one business workflow. A single up command starts containers but does not replace key provisioning, migrations, domain seeds and account-link setup.

Evidence: readme.md

C.5.8 OPS-02 – CI quality and image builds

Status: **Partial** **Priority:** Must

Requirement. Pushes to main shall trigger independent lint, type, test, contract and container build checks, including a 70-percent coverage gate.

Acceptance. Observe a real controller-triggered successful and failing build, retain JUnit/coverage artifacts and exact commit. Definitions exist; webhook/job configuration and successful live execution are not verified here. Image publication is not implemented in root CI.

Evidence: Jenkinsfile

C.5.9 OPS-03 – Controlled promotion

Status: **Partial** **Priority:** Must

Requirement. Production promotion shall use an explicitly selected tested release, candidate verification, human approval, backup checks, migrations, smoke validation and release recording.

Acceptance. Validate the actual Jenkins controller and artifact supply chain. Confirm parameters, effective Compose environment and secret files. Rollback shall be human-authorized and schema-compatible; automatic destructive down-migration is excluded.

Evidence: ops/jenkins/Deploy.Jenkinsfile

C.5.10 OPS-04 – Load acceptance

Status: **Partial** **Priority:** Must

Requirement. The agreed read workload shall meet p95 below 500 ms and error rate below one percent for 25 concurrent sessions over five minutes on a representative host.

Acceptance. Repeat the defined workload with recorded dataset, pacing and hardware. The existing 90-second result fails the latency target and is insufficient for full-duration acceptance.

Evidence: docs/performance.md

C.5.11 OPS-05 – Logging and monitoring

Status: **Partial** **Priority:** Must

Requirement. Operators shall have centralized diagnostic logs, liveness/readiness checks and actionable queue/resource/backup alerts.

Acceptance. Existing script checks process liveness, queue backlog over 100, any DLQ messages, disk at 85 percent and memory at 90 percent by default. Central collection, worker heartbeat and backup-age alerts require additional verified integration.

Evidence: ops/monitoring/health_check.py

C.5.12 OPS-06 – Horizontal scaling strategy

Status: **Target** **Priority:** Must

Requirement. The technical design shall specify safe API/worker scaling, connection budgets and state placement even where actual replication is not demonstrated.

Acceptance. Review the SDD scaling view, duplicate-settlement protection and shared-cluster bottlenecks. A conceptual diagram shall not be counted as measured HA or replica capacity.

Evidence: docs/specifications/sdd.tex

C.5.13 OPS-07 – Security mitigation evidence

Status: **Partial** **Priority:** Must

Requirement. At least five applicable OWASP risk categories shall have code-level mitigation and repeatable tests, with configuration limitations explicitly recorded.

Acceptance. Trace access control, credential protection, injection prevention, safe financial design, gateway hardening and logging to tests/configuration. Reconcile legacy security prose with actual bootstrap and Compose settings before asserting production controls.

Evidence: docs/security.md

C.5.14 DOC-01 – Submission and traceability

Status: **Partial** **Priority:** Must

Requirement. The submission shall include the SRS, technical design/UML, source history, setup/API guide, test/coverage reports, issue tracking and signed individual/integrity statements.

Acceptance. Compile both documents, render the diagram suite and verify code consistency. Team must add actual signatures, contribution statements, run evidence and a fallback video at most five minutes long; these cannot be fabricated by documentation generation.

Evidence: docs/specifications/README.md

C.6 Frontend interaction requirements

C.6.1 UI-01 – Session-aware navigation

Status: **Implemented** **Priority:** Must

Requirement. The SPA shall restore sessions on startup, guard protected pages and return a successfully authenticated user to the requested pathname.

Acceptance. Loading shows a status indicator; guest navigation redirects to login. Login uses Zod email/password validation; invalid local input shall not call the API. Refresh is attempted once per failed request sequence and deduplicated within one runtime.

Evidence: ../ERP_System/src/features/auth/AuthContext.tsx

C.6.2 UI-02 – Live student academic workflow

Status: **Implemented** **Priority:** Must

Requirement. Students shall enroll, observe pending invoice creation, register compatible offerings, view own published grades, submit appeals and download academic PDFs.

Acceptance. A student without campus context sees an explicit error. A new enrollment invalidates the enrollment query and polls its invoice every two seconds until found. Registration refreshes own registrations and risk data. Instructor controls are not present.

Evidence: ../ERP_System/src/features/academic/EnrollmentPage.tsx

C.6.3 UI-03 – Payment progress and receipts

Status: **Partial** **Priority:** Must

Requirement. The student payment screen shall display pending, failed and succeeded intent states, allow retry after failure and expose receipts only for paid invoices.

Acceptance. Current screen polls every 1.5 seconds while the fetched intent is pending and invalidates invoice data on success. Payment-poll error feedback and recovery of an in-progress intent after a page reload need additional acceptance work.

Evidence: ../ERP_System/src/features/finance/InvoiceRow.tsx

C.6.4 UI-04 – Employee workspace

Status: **Implemented** **Priority:** Must

Requirement. A staff user shall submit leave, enter a signed attendance code, view approved payslips and read own notifications through live API-backed sections.

Acceptance. Successful mutations invalidate their matching queries. Manual token entry is the implemented QR interaction; no camera scanner or biometric hardware is required for this baseline. New leave decisions are not pushed in real time to an already open page.

Evidence: ../ERP_System/src/features/hr/HRPage.tsx

C.6.5 UI-05 – Responsive and accessible presentation

Status: **Partial** **Priority:** Must

Requirement. The frontend shall support keyboard use, readable status/error states, reduced motion and layouts without document-level horizontal overflow at representative device sizes.

Acceptance. Existing Playwright assets check 360x800, 768x1024, 1440x900 and 1920x1080, axe serious/critical issues and console failures for selected pages. This does not certify complete WCAG compliance or all role-specific workflows; manual keyboard/mobile testing remains required.

Evidence: `../ERP_System/tests/e2e/authenticated.spec.ts`

C.6.6 UI-06 – Client data lifecycle and truthful states

Status: **Partial** **Priority:** Must

Requirement. The UI shall distinguish empty, pending, failed and unavailable records and shall not retain another authenticated principal's cached domain data after an account change.

Acceptance. Skeleton, empty and error components exist. Query keys currently omit user/tenant identity and auth transitions do not clear the shared cache; isolation on account switch requires implementation/verification. Download and nested-query failures also need explicit UI checks.

Evidence: `../ERP_System/src/main.tsx`

D Detailed use-case specifications

D.1 UC-01 – Authenticate and maintain a session

Primary actor: any active user. **Related:** AUTH-01–05. **Precondition:** configured keys and migrated Identity database; account exists.

1. User submits email/password to the public login endpoint.
2. Identity locates account using platform credential-resolution context, checks active/lockout status and verifies Argon2 hash.
3. Identity clears failure count, persists a refresh-session hash and returns access JWT plus refresh cookie.
4. Client sends bearer token for protected operations; gateway delegates verification, and the target service verifies the token again.
5. At access expiry the client refreshes using the cookie and replaces its access token.

Alternatives: bad credentials increment failures; lockout blocks login; expired/missing refresh returns 401; reuse revokes the family. Concurrent refreshes need explicit client/server acceptance. **Postcondition:** successful session is identified by claims, not client tenant headers. **No promise:** immediate invalidation of issued access tokens on logout.

D.2 UC-02 – Enroll and obtain a tuition invoice

Primary actor: student or authorized admin/staff. **Related:** ACAD-02, FIN-01–03, INT-03. **Precondition:** valid program/term and separately configured matching fee.

1. Caller chooses program/term/campus; staff supplies a student ID while student identity is derived from the token.
2. Academic validates local references and duplicates, then commits enrollment and outbox.
3. HTTP 201 confirms academic acceptance without waiting for Finance.
4. Worker publishes the event with persistent delivery and publisher confirms.
5. Finance consumer inserts inbox ID, finds fee schedule and inserts pending invoice in one transaction.
6. Student polls invoice list by enrollment and views the pending invoice.

Alternatives: broker down leaves outbox unpublished; missing fee sends message to DLQ and rolls back inbox; transient database failure requeues; duplicate event/enrollment is acknowledged without another invoice. **Postcondition:** one accepted enrollment and eventually one invoice effect when required configuration and infrastructure are healthy.

D.3 UC-03 – Register a course with prerequisites

Primary actor: student. **Related:** ACAD-03–04. **Precondition:** own accepted enrollment, published prerequisite results.

1. Student selects offering and enrollment.
2. Academic checks ownership, term and course program.
3. Each prerequisite is satisfied by a published percentage average of at least 50 in any offering of that prerequisite course.
4. Academic rejects unmet prerequisites or duplicate registration, otherwise commits 201.

Alternatives: missing/foreign enrollment yields 404; mismatch yields 400; unmet prerequisites yields 409 with missing course IDs. **Postcondition:** registration links student, offering and enrollment. Capacity limiting is not currently enforced.

D.4 UC-04 – Publish and appeal a grade

Actors: assigned instructor/admin and student. **Related:** ACAD-05,07,08.

1. Instructor creates an assessment, enters bounded scores and explicitly publishes.
2. Student retrieves an own published grade and submits an appeal with a reason.
3. Authorized reviewer moves appeal under review or decides directly.
4. Acceptance supplies corrected score; changed values append grade audit history.

5. Appeal decision, metadata and correction commit together.

Alternatives: unpublished/foreign grade cannot be appealed; duplicate open appeal is rejected; final states cannot reopen; correction beyond maximum is rejected. **Postcondition:** decision is persisted and, if changed through appeal, score history is recorded. Ordinary grade edits remain outside this audit guarantee.

D.5 UC-05 – Settle tuition and retrieve receipt

Actor: student; supporting actor is the selected payment provider. **Related:** FIN-03–07.

1. Caller selects a pending owned invoice and submits payer mobile number.
2. Finance commits intent/reference, then calls request-to-pay.
3. Client GET or three-second worker polling asks provider status.
4. Pending leaves business state unchanged; failed marks intent failed.
5. Success locks intent and invoice, marks paid, posts balanced entries and adds receipt, then commits all changes.
6. Caller downloads receipt PDF.

Alternatives: provider timeout leaves pending, not paid; GET errors may surface; duplicate reconciliation is a no-op for terminal intent. A callback is only a hint to query provider status; current gateway prevents unauthenticated provider ingress. **Postcondition:** paid status is backed by ledger and receipt, not by browser assertions.

D.6 UC-06 – Schedule an exam safely

Actor: authorized instructor/admin. **Related:** ACAD-09. Actor supplies offering, room and start/end. Back-end verifies authority and interval, locks institution/term scheduling, tests room, instructor and shared-student overlap, then persists or returns 409. Rescheduling excludes the current record from conflicts. The submitted offering field does not change the existing offering during reschedule. Adjacent intervals are allowed; cross-term overlap is outside the implemented search.

D.7 UC-07 – Request leave and notify the employee

Actors: linked employee and different HR approver. **Related:** HR-04–05. Employee submits dates/reason; HR sees pending request and approves or rejects; backend rejects self-approval and already-decided requests, then writes decision and in-app notification in one commit. The employee polls notifications and marks the notice read. Unlinked employees cannot use self-service. There is no email delivery guarantee, automatic quota enforcement or working-day calculation.

D.8 UC-08 – Generate and approve payroll

Actors: HR administrator and rate-verifying super administrator. **Related:** HR-07–09. An HR administrator creates an unverified rate version and generates a draft period run for active employees. Calculation uses the selected version, not an implicit statutory default. A super administrator checks authoritative sources and attests the schedule. HR approves the draft only after verification; employees retrieve their own saved payslips. A repeated period fails and unverified approval returns 403. Approval records a decision; it does not transfer money.

D.9 UC-09 – Record employee attendance

Actors: HR administrator and linked employee. **Related:** HR-03. HR creates a shift and issues a token through the API; the employee UI accepts the token as pasted or typed text. Issuing another token replaces the stored token ID. Employee submits token with an ERP bearer session; backend checks QR signature/expiry/current ID, shift ownership and uniqueness before recording check-in. Expiry/replacement yields 401; repeated check-in yields 409. Camera scanning and QR image issuance screens are not implemented.

D.10 UC-10 – Control stock movements

Actor: HR administrator. **Related:** HR-10. Actor selects asset, delta, optional employee and reason. Back-end locks asset, calculates aggregate assignments and rejects a movement that exceeds stock or reduces total

stock below assignments. Employee-linked deltas alter assignment balance; unlinked deltas alter total quantity. Commit preserves movement history. Employee-specific over-return prevention needs additional rules beyond the aggregate check.

E Requirement-to-verification traceability

Test file names below are existing assets. “Additional” denotes a planned acceptance scenario, not a newly implemented test. All service test paths start at `services/<service>/tests/`. The companion SDD uses the same requirement IDs.

Requirement IDs	Existing verification assets	Additional acceptance / scope
AUTH-01-05	<code>identity/test_auth.py</code>	Browser expiry/logout and concurrent refresh race; exact lockout/time boundaries.
AUTH-02,06,07	<code>academic/test_me.py</code> ; <code>finance/test_me.py</code> ; <code>hr/test_me.py</code>	Spoofed headers through gateway; UI guards; platform-admin boundary.
AUTH-07, DATA-01	<code>tests/integration/test_db_isolation.py</code>	Two-tenant checks for RLS-protected tables; inspect metadata exceptions.
AUTH-08,09	Config inspection; <code>hr/test_config.py</code>	Effective production cookie/TLS validation; MFA absent.
ACAD-01,03,04	<code>academic/test_courses.py</code>	Missing CRUD; graph/registration concurrency; capacity policy decision.
ACAD-02, FIN-02	<code>academic/test_enrollments.py</code> ; <code>finance/test_consumer.py</code>	Broker outage, restart after publish-before-mark and UI eventual consistency.
ACAD-05,06	<code>academic/test_attendance.py</code>	Instructor A versus unrelated instructor B; missing marks and zero sessions.
ACAD-07,08	<code>academic/test_grades.py</code>	Final-state redecision, duplicate open appeal under concurrency, audit scope.
ACAD-09	<code>academic/test_exams.py</code>	Concurrent same-term scheduling, boundary adjacency and explicit cross-term limitation.
ACAD-10,11	<code>academic/test_reports.py</code>	Latest-two grade ordering, exact thresholds, UI risk explanation and actual PDF content.
FIN-01,03	<code>finance/test_fee_schedules.py</code>	Foreign-tenant IDs, student invoice concealment and fee prerequisites.
FIN-04-06	<code>finance/test_payments.py</code> ; <code>finance/test_gateway.py</code>	Real gateway callback path; authorized MTN sandbox run; crash before provider request.
FIN-05	<code>finance/test_reconciliation.py</code>	Concurrent callback/GET/poller settlement and transaction rollback injection.
FIN-07	<code>finance/test_receipt.py</code>	Paid/unknown/foreign invoice PDF access through gateway.
FIN-08	<code>finance/test_expenses.py</code>	Balanced ledger sum by transaction and rollback on failed posting.
FIN-09,10	<code>finance/test_campaigns.py</code>	Repeated student attribution, zero-cost null ROI and date policy.
FIN-11,12	<code>finance/test_summaries.py</code> ; <code>finance/test_payment_worker.py</code>	Missed month catch-up; regeneration version history; UI dashboards.
HR-01,02	<code>hr/test_recruitment.py</code>	Concurrent hires and manually linked identity lifecycle; absent employee CRUD.
HR-03	<code>hr/test_attendance.py</code> ; <code>hr/test_config.py</code>	Cross-token audience confusion, QR replacement and frontend scan path.
HR-04,05	<code>hr/test_leave.py</code>	Concurrent decisions, in-app delivery, own notification read isolation.
HR-06,11	<code>hr/test_performance.py</code>	Dashboard aggregation and no-profile/foreign-profile access.
HR-07,08	<code>hr/test_payroll_calc.py</code>	Signed official rate worksheet; monthly/annual conventions; half-up rounding.
HR-09	<code>hr/test_payroll_run.py</code>	Repeated approval, selected effective period and absence of fund transfer.
HR-10	<code>hr/test_assets.py</code>	Concurrent assignment; aggregate versus employee-specific returns.
DATA-02, OPS-04	<code>tests/load/k6-smoke.js</code> ; performance report	Full five-minute representative run and before/after SQL plans.
DATA-03	<code>ops/backups/pitr_drill.ps1</code>	Multi-database restore, off-host archive, representative RPO/RTO.

Requirement IDs	Existing verification assets	Additional acceptance / scope
INT-01,02	Gateway configuration; service health tests	API export, gateway budget and auth-negative tests.
INT-03	<code>contracts/test_examples_match_schema.py</code>	Runtime unknown type/version, malformed non-object envelope and DLQ replay.
OPS-01-03	Root and deployment Jenkins definitions	Clean-host setup, live controller validation, artifact push and promotion evidence.
OPS-05-07	Monitoring script; security report	Central log collector, backup alerting, replica behavior and security-negative test evidence.
DOC-01	This package and PlantUML sources	Group sign-off, issue-board export, signed statements and fallback video.
UI-01	Frontend <code>LoginPage.test.tsx</code> ; <code>ProtectedRoute.test.tsx</code>	Session restoration, multi-tab rotation, logout error and return pathname.
UI-02	Frontend <code>EnrollmentPage.test.tsx</code> ; <code>CoursesPanel.test.tsx</code>	Broker-driven live invoice appearance, report downloads and instructor UI gap.
UI-03	Frontend <code>FinancePage.test.tsx</code>	Poll failures, reload during pending intent and actual sandbox settlement.
UI-04	Frontend <code>HRPage.test.tsx</code>	Linked-employee end-to-end check-in, leave and notification delivery.
UI-05	Frontend <code>tests/e2e/login.spec.ts</code> ; <code>authenticated.spec.ts</code>	Mobile navigation/logout, focus, all roles and manual accessibility.
UI-06	Shared frontend query/client code	Account switch cache purge; nested-grade errors; download/mark-read failures.

E.1 Critical acceptance scenario set

- AT-1 **Tenant escape negative test:** seed tenant A/B, use A's token with B's resource ID and spoofed headers; verify no B data is returned or modified.
- AT-2 **Broker outage:** disable only a disposable test broker after setup; enroll; verify committed outbox and no synchronous dependency; restore broker and obtain one invoice.
- AT-3 **Consumer replay:** deliver duplicate event ID and duplicate enrollment under a new event ID; verify one invoice and visible missing-fee DLQ behavior.
- AT-4 **Settlement atomicity:** reconcile the same success from two callers, inject failure before commit and inspect intent/invoice/receipt/ledger invariants after retry.
- AT-5 **Exam conflicts:** schedule overlapping room, instructor and common-student exams; verify rejection and adjacent-slot acceptance under simultaneous requests.
- AT-6 **Payroll release gate:** create an unverified schedule, generate a draft, attempt approval, attest using reviewed source evidence, then approve.
- AT-7 **Recovery:** restore an isolated representative ERP cluster to a known time, reconcile queue/outbox state and revoke obsolete sessions before service restoration.
- AT-8 **Production configuration:** inspect effective merged settings and externally observed cookie/TLS/listener behavior; do not rely on comments in override files.

F Use-case and context diagrams

The diagram sources are shared with the SDD. Detailed sequence, state, ERD and deployment views are in that document to avoid duplicating the entire suite.

F.1 System context and external responsibilities

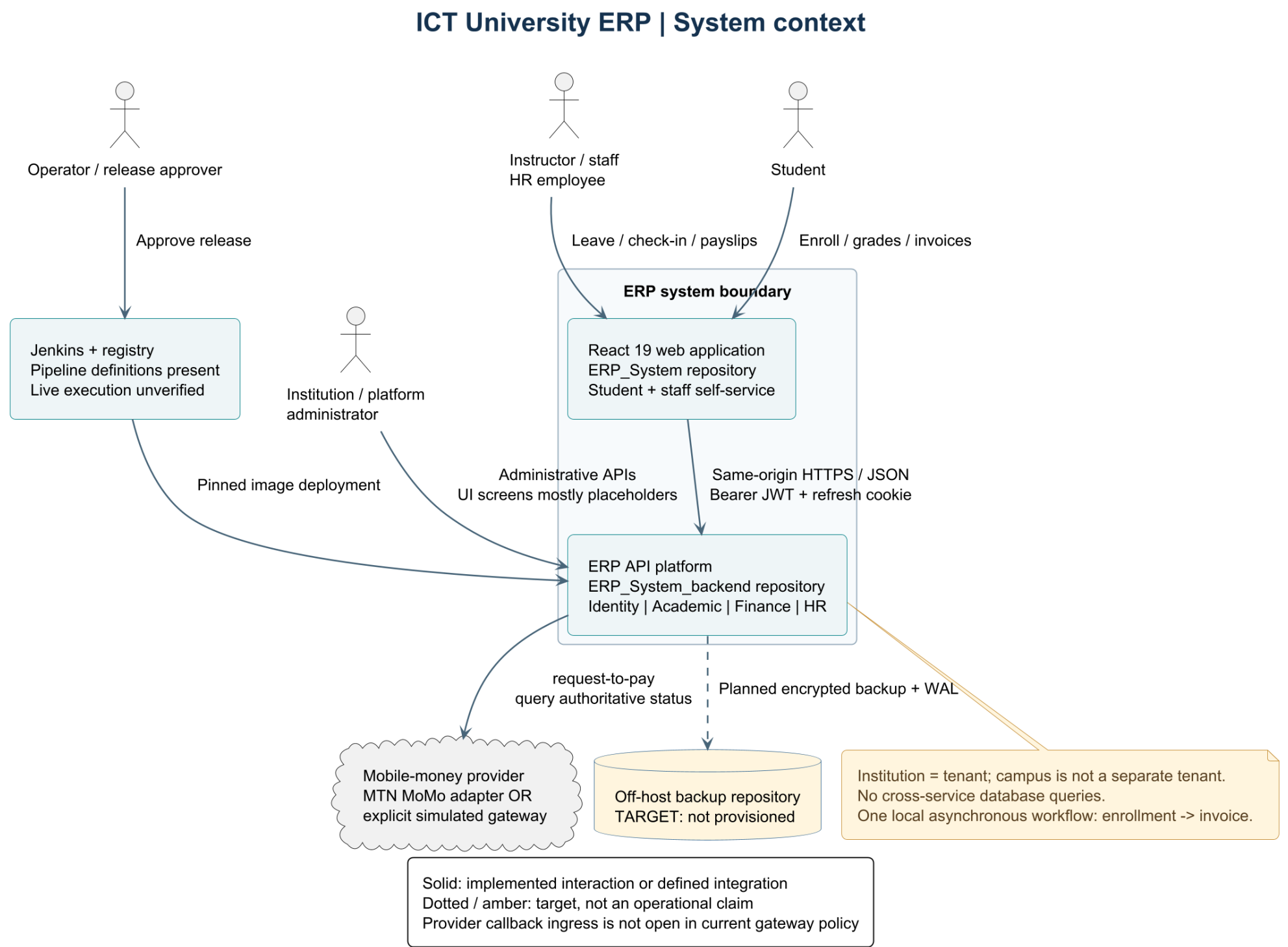


Figure 1: System context and external responsibilities. The frontend and backend are within the product boundary; provider and operating dependencies are explicit.

Editable source: `diagrams/01-context.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

F.2 Core actor and use-case model

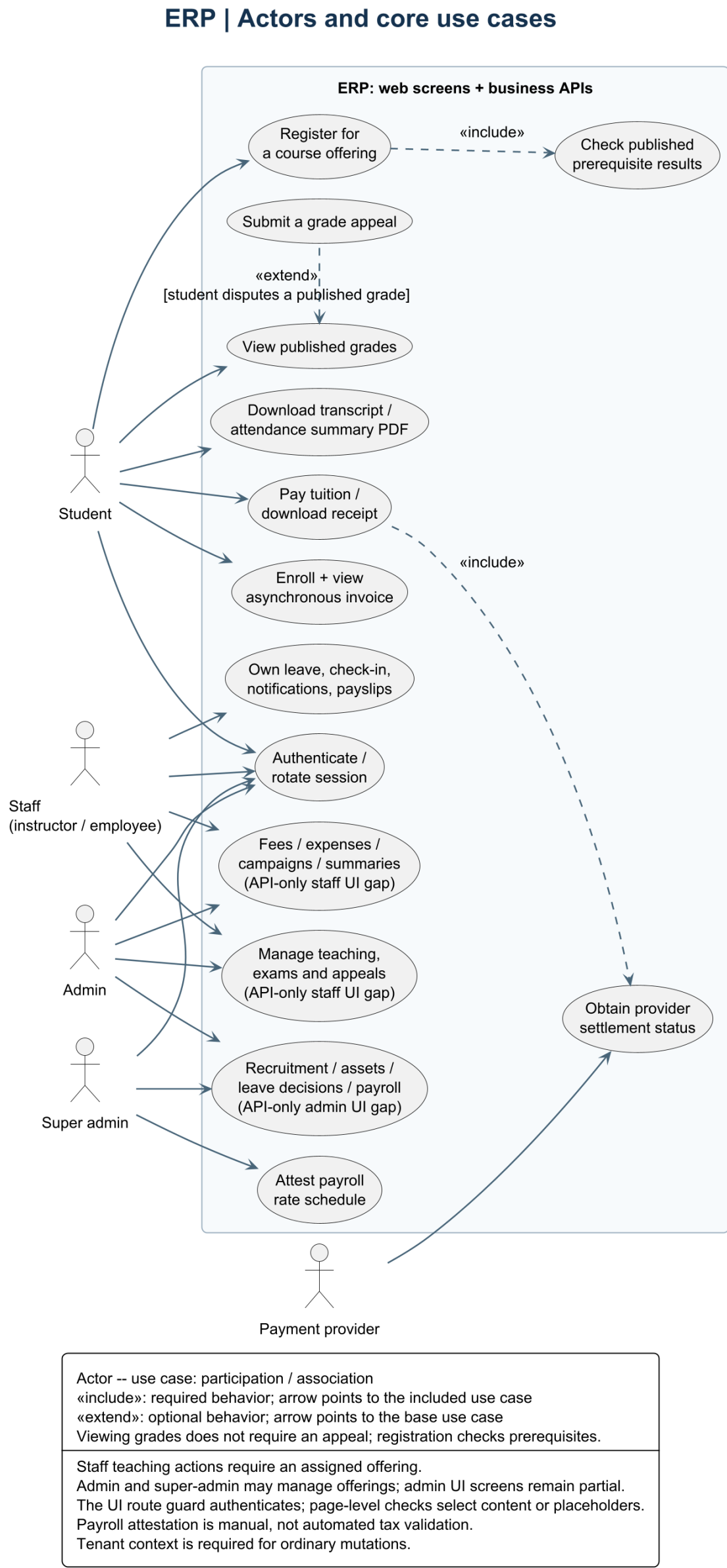


Figure 2: Core actor and use-case model. Business responsibilities map to implemented roles; target UI and provisioning work is identified separately.

Editable source: `diagrams/02-use-cases.puml`. This is a large-format diagram sheet; use original size or the companion SVG for full-detail review. Solid relationships denote implemented structure unless otherwise marked; dashed notes distinguish logical, external or target elements.

G Glossary and submission checklist

Term	Meaning in this project
Tenant / institution	Primary data isolation boundary carried in verified claims.
Campus	An institution location referenced by identity/enrollment; not a separate tenant.
Offering	Course taught in a term with an assigned instructor, room and stored capacity.
Enrollment / registration	Enrollment admits a student to program/term; registration selects an offering.
Outbox / inbox	Producer transaction record / consumer duplicate-event record.
DLQ	Dead-letter queue for rejected events requiring operator investigation.
At-least-once	Delivery may repeat; business idempotency prevents duplicate effects.
XAF / FCFA	Central African CFA franc; whole-franc integer amounts in this backend.
CNPS / IRPP	Social-insurance contribution / personal-income-tax calculation parameters.
RPO / RTO	Maximum tolerable lost-data interval / target restoration duration.
RLS	PostgreSQL row-level security using transaction-local institution context.
PITR / WAL	Point-in-time recovery / write-ahead log.
Evidence-grounded	Supported by identified source or artifact, without inventing run results.

G.1 Team-owned completion checklist

- Review and approve every partial/target requirement; resolve or disclose its impact.
- Attach real test, coverage, performance and recovery evidence with commit IDs.
- Export OpenAPI from the identified running service versions.
- Provide issue-tracker status with resolved/open items and complete Git history.
- Verify the clean setup guide including sibling frontend and migrations/seeds.
- Complete four one-page contribution statements and a jointly signed integrity declaration.
- Disclose AI-assisted documentation/diagrams and any reused code at the defense.
- Prepare the five-minute live demonstration and optional fallback video of at most five minutes.

G.2 Source references

1. Supplied SEN4121 assignment, `docs/Large_System_Environment_SET_A_Summer2026.pdf`, pages 1–4; summarized, not reproduced.
2. Backend implementation baseline `6e10c54`, service models, routers, migrations and tests identified throughout this specification.
3. Frontend `../ERP_System`, baseline `6cc6859`, React routes/components, service adapters, Vitest and Playwright assets identified throughout this specification.
4. `docs/performance.md`, historical 25-VU local smoke disclosure.
5. `docs/backup-recovery.md`, isolated 12 September 2026 PITR disclosure.
6. `docs/security.md`, supporting historical control narrative, qualified where bootstrap and Compose evidence differ.
7. ISO/IEC/IEEE 29148:2018, *Systems and software engineering – Life cycle processes – Requirements engineering*; organizational reference only.